

Практическая работа №4 ООП в Java. Исключения

Кармаин Андрей Александрович · группа ИНБ-30-25 · 21.09.2026

Исходники: github.com/akarmain/vanilla_code/tree/main/MIREA/trem_2/Java/Workbook_4

Задание №1: класс Triangle от GeometricObject, getArea(), getPerimeter(), toString()

0_класс_triangle_от_geometricobject.java

```
// Задание №1: класс Triangle от GeometricObject, getArea(), getPerimeter(), toString()
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Введите три стороны треугольника: ");
        double s1 = input.nextDouble();
        double s2 = input.nextDouble();
        double s3 = input.nextDouble();

        System.out.print("Введите цвет: ");
        String color = input.next();

        System.out.print("Треугольник закрашен (true/false)? ");
        boolean filled = input.nextBoolean();

        Triangle t = new Triangle(s1, s2, s3);
        t.setColor(color);
        t.setFilled(filled);

        System.out.println(t);
        System.out.println("Площадь: " + t.getArea());
        System.out.println("Периметр: " + t.getPerimeter());
        System.out.println("Цвет: " + t.getColor());
        System.out.println("Закрашен: " + t.isFilled());
    }
}

class GeometricObject {
    private String color = "белый";
    private boolean filled;

    public GeometricObject() {
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }

    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }
}

class Triangle extends GeometricObject {
    private double side1 = 1.0;
    private double side2 = 1.0;
    private double side3 = 1.0;

    public Triangle() { // безаргументный: стороны по умолчанию 1.0
    }

    public Triangle(double side1, double side2, double side3) {
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }
}
```

```

public double getSide1() { return side1; }
public double getSide2() { return side2; }
public double getSide3() { return side3; }

public double getPerimeter() {
    return side1 + side2 + side3;
}

public double getArea() {
    double s = getPerimeter() / 2; // формула Герона
    return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3)); // полупериметр
}

@Override
public String toString() {
    return "Треугольник: сторона1 = " + side1 + " сторона2 = " + side2 + " сторона3 = " + side3;
}
}

```

Входные данные: 3 4 5 ↵ синий ↵ true

Выходные данные:

```

Введите три стороны треугольника: Введите цвет: Треугольник закрашен (true/false)? Треугольник: сторона1
= 3.0 сторона2 = 4.0 сторона3 = 5.0
Площадь: 6.0
Периметр: 12.0
Цвет: синий
Закрашен: true

```

Задание №2: класс `IllegalTriangleException`, проверка неравенства треугольника в конструкторе

1_исключение_illegaltriangleexception.java

```

// Задание №2: класс IllegalTriangleException, проверка неравенства треугольника в конструкторе
public class Main {
    public static void main(String[] args) {
        try {
            Triangle ok = new Triangle(3, 4, 5);
            System.out.println(ok + ", площадь " + ok.getArea());

            Triangle bad = new Triangle(1, 2, 10); // 1 + 2 < 10 - треугольника не существует
            System.out.println(bad);
        } catch (IllegalTriangleException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
    }
}

class IllegalTriangleException extends Exception { // своё проверяемое исключение
    public IllegalTriangleException(String message) {
        super(message);
    }
}

class GeometricObject {
    private String color = "белый";
    private boolean filled;

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }

    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }
}

```

```

class Triangle extends GeometricObject {
    private double side1 = 1.0;
    private double side2 = 1.0;
    private double side3 = 1.0;

    public Triangle() {
    }

    /** Создает треугольник с указанными сторонами */
    public Triangle(double side1, double side2, double side3) throws IllegalArgumentException {
        if (side1 + side2 <= side3 || side1 + side3 <= side2 || side2 + side3 <= side1) {
            throw new IllegalArgumentException(
                "сумма двух сторон должна быть больше третьей: " + side1 + ", " + side2 + ", " + side3);
        }
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    public double getSide1() { return side1; }
    public double getSide2() { return side2; }
    public double getSide3() { return side3; }

    public double getPerimeter() { return side1 + side2 + side3; }

    public double getArea() {
        double s = getPerimeter() / 2;
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    @Override
    public String toString() {
        return "Треугольник: сторона1 = " + side1 + " сторона2 = " + side2 + " сторона3 = " + side3;
    }
}

```

Входные данные: не требуется

Выходные данные:

Треугольник: сторона1 = 3.0 сторона2 = 4.0 сторона3 = 5.0, площадь 6.0
 Ошибка: сумма двух сторон должна быть больше третьей: 1.0, 2.0, 10.0

Задание №3: GeometricObject реализует Comparable, статический max(), ComparableCircle

2_comparable_и_метод_max.java

```

// Задание №3: GeometricObject реализует Comparable, статический max(), ComparableCircle
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Введите три стороны треугольника: ");
        Triangle t = new Triangle(input.nextDouble(), input.nextDouble(), input.nextDouble());
        System.out.print("Введите цвет: ");
        t.setColor(input.next());
        System.out.print("Закрашен (true/false)? ");
        t.setFilled(input.nextBoolean());

        System.out.println(t);
        System.out.println("Площадь " + t.getArea() + ", периметр " + t.getPerimeter()
            + ", цвет " + t.getColor() + ", закрашен " + t.isFilled());
    }
}

```

```

        Circle c1 = new Circle(2), c2 = new Circle(5);
        Rectangle r1 = new Rectangle(2, 3), r2 = new Rectangle(4, 4);

        System.out.println("Больший круг: " + GeometricObject.max(c1, c2));
        System.out.println("Больший прямоугольник: " + GeometricObject.max(r1, r2));

        ComparableCircle cc1 = new ComparableCircle(1), cc2 = new ComparableCircle(7);
        System.out.println("Больший ComparableCircle: "
            + (cc1.compareTo(cc2) > 0 ? cc1 : cc2));
        System.out.println("Круг против прямоугольника: "
            + (cc2.compareTo(r2) > 0 ? cc2 : r2));
    }
}

abstract class GeometricObject implements Comparable<GeometricObject> {
    private String color = "белый";
    private boolean filled;

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }

    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    public abstract double getArea();

    @Override
    public int compareTo(GeometricObject other) { // сравнение по площади
        return Double.compare(this.getArea(), other.getArea());
    }

    public static GeometricObject max(GeometricObject a, GeometricObject b) {
        return a.compareTo(b) >= 0 ? a : b;
    }
}

class Circle extends GeometricObject {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    public double getRadius() { return radius; }

    @Override
    public double getArea() { return Math.PI * radius * radius; }

    @Override
    public String toString() { return "Круг r=" + radius + " (площадь " + getArea() + ")"; }
}

class ComparableCircle extends Circle {
    public ComparableCircle(double radius) { super(radius); }

    @Override
    public String toString() { return "ComparableCircle r=" + getRadius() + " (площадь " + getArea() + ")"; }
}

class Rectangle extends GeometricObject {
    private double width;
    private double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double getArea() { return width * height; }
}

```

```

@Override
public String toString() { return "Прямоугольник " + width + "x" + height + " (площадь " + getArea()
+ ")"; }
}

class Triangle extends GeometricObject {
    private double side1 = 1.0, side2 = 1.0, side3 = 1.0;

    public Triangle() { }

    public Triangle(double side1, double side2, double side3) {
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    public double getPerimeter() { return side1 + side2 + side3; }

    @Override
    public double getArea() {
        double s = getPerimeter() / 2;
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    @Override
    public String toString() {
        return "Треугольник: сторона1 = " + side1 + " сторона2 = " + side2 + " сторона3 = " + side3;
    }
}

```

Входные данные: 3 4 5 ↵ синий ↵ true

Выходные данные:

Введите три стороны треугольника: Введите цвет: Закрашен (true/false)? Треугольник: сторона1 = 3.0
 сторона2 = 4.0 сторона3 = 5.0
 Площадь 6.0, периметр 12.0, цвет синий, закрашен true
 Большой круг: Круг r=5.0 (площадь 78.53981633974483)
 Большой прямоугольник: Прямоугольник 4.0x4.0 (площадь 16.0)
 Большой ComparableCircle: ComparableCircle r=7.0 (площадь 153.93804002589985)
 Круг против прямоугольника: ComparableCircle r=7.0 (площадь 153.93804002589985)

Задание №4: интерфейс Colorable с howToColor(), класс Square, массив из пяти GeometricObject

3_интерфейс_colorable_класс_square.java

```

// Задание №4: интерфейс Colorable с howToColor(), класс Square, массив из пяти GeometricObject
public class Main {
    public static void main(String[] args) {
        GeometricObject[] objects = {
            new Square(3),
            new Circle(2),
            new Square(1.5),
            new Rectangle(2, 5),
            new Circle(4)
        };

        for (GeometricObject o : objects) {
            System.out.println(o + ", площадь " + o.getArea());
            if (o instanceof Colorable) {
                ((Colorable) o).howToColor();
            }
        }
    }
}

```

```

interface Colorable {
    void howToColor();
}

abstract class GeometricObject {
    private String color = "белый";
    private boolean filled;

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }

    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    public abstract double getArea();
}

class Square extends GeometricObject implements Colorable {
    private double side;

    public Square() { this.side = 0; }
    public Square(double side) { this.side = side; }

    public double getSide() { return side; }
    public void setSide(double side) { this.side = side; }

    @Override
    public double getArea() { return side * side; }

    @Override
    public void howToColor() {
        System.out.println("Раскрасьте все четыре стороны.");
    }

    @Override
    public String toString() { return "Квадрат со стороной " + side; }
}

class Circle extends GeometricObject {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public double getArea() { return Math.PI * radius * radius; }

    @Override
    public String toString() { return "Круг r=" + radius; }
}

class Rectangle extends GeometricObject {
    private double width, height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double getArea() { return width * height; }

    @Override
    public String toString() { return "Прямоугольник " + width + "x" + height; }
}

```

Входные данные: не требуется

Выходные данные:

Квадрат со стороной 3.0, площадь 9.0
Раскрасьте все четыре стороны.
Круг r=2.0, площадь 12.566370614359172
Квадрат со стороной 1.5, площадь 2.25
Раскрасьте все четыре стороны.
Прямоугольник 2.0x5.0, площадь 10.0
Круг r=4.0, площадь 50.26548245743669

Задание 1 (исключения): два массива, ввод числа 1..12, перехват `ArrayIndexOutOfBoundsException`

`4_месяцы_дни_массивы_исключение.java`

```
// Задание 1 (исключения): два массива, ввод числа 1..12, перехват ArrayIndexOutOfBoundsException
import java.util.InputMismatchException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        String[] months = {"январь", "февраль", "март", "апрель", "май",
                           "июнь", "июль", "август", "сентябрь", "октябрь", "ноябрь", "декабрь"};
        int[] dom = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        Scanner input = new Scanner(System.in);

        while (true) {
            // повторяем, пока не введут корректное число
            System.out.print("Введите номер месяца (1-12): ");
            try {
                int n = input.nextInt();
                System.out.println(months[n - 1] + " — " + dom[n - 1] + " дней");
                break;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Недопустимое число");
            } catch (InputMismatchException e) {
                System.out.println("Нужно ввести целое число");
                input.next();
            }
        }
    }
}
```

Входные данные: 13 ↵ abc ↵ 5

Выходные данные:

Введите номер месяца (1-12): Недопустимое число
Введите номер месяца (1-12): Нужно ввести целое число
Введите номер месяца (1-12): май — 31 дней

Задание 2 (исключения): ввод года для февраля, метод проверки високосного года

`5_февраль_високосный_год.java`

```
// Задание 2 (исключения): ввод года для февраля, метод проверки високосного года
import java.util.InputMismatchException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        String[] months = {"январь", "февраль", "март", "апрель", "май",
                           "июнь", "июль", "август", "сентябрь", "октябрь", "ноябрь", "декабрь"};
        int[] dom = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        Scanner input = new Scanner(System.in);
```

```

while (true) {
    System.out.print("Введите номер месяца (1-12): ");
    try {
        int n = input.nextInt();
        int days = dom[n - 1];
        String name = months[n - 1];

        if (n == 2) { // февраль – спрашиваем год
            System.out.print("Введите год: ");
            int year = input.nextInt();
            if (isLeapYear(year)) {
                days = 29;
            }
            System.out.println(name + " " + year + " года – " + days + " дней");
        } else {
            System.out.println(name + " – " + days + " дней");
        }
        break;
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Недопустимое число");
    } catch (InputMismatchException e) {
        System.out.println("Нужно ввести целое число");
        input.next();
    }
}

/** Високосный: делится на 4, но не на 100; либо делится на 400 */
public static boolean isLeapYear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}
}

```

Входные данные: 2 ↵ 2024

Выходные данные:

Введите номер месяца (1-12): Введите год: февраль 2024 года – 29 дней